

1. Data Wrangling

Refer “heart_disease” and perform following data wrangling operations

1. Scan all variables for missing values and inconsistencies. If there are missing values and/or inconsistencies, use any of the suitable techniques to deal with them.
2. Scan all numeric variables for outliers. If there are outliers, use any of the suitable techniques to deal with them.
3. Apply data transformations on at least one of the variables. The purpose of this transformation should be one of the following reasons: to change the scale for better understanding of the variable, to convert a non-linear relation into a linear one, or to decrease the skewness and convert the distribution into a normal distribution.

```
# Import necessary libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from scipy import stats
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Step 1: Load the heart disease dataset
```

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/heart-disease.data" # UCI Heart Disease dataset URL
```

```
columns = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
```

```
df = pd.read_csv(url, names=columns)
```

```
# Step 2: Scan for missing values
```

```
print("\nMissing Values in Each Column:")
```

```
print(df.isnull().sum())
```

```

# Step 3: Check for inconsistencies in categorical variables

# Checking for unique values in categorical variables like 'sex', 'cp', etc.
print("\nUnique Values in Each Column:")
print(df.nunique())

# Step 4: Handle missing values and inconsistencies

# Since there are no missing values in this dataset, we'll just move on.
# If there were missing values, we could use df.fillna() or df.dropna()

# Step 5: Scan for outliers in numeric variables using IQR
numeric_columns = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']

# Calculate Q1 (25th percentile) and Q3 (75th percentile)
Q1 = df[numeric_columns].quantile(0.25)
Q3 = df[numeric_columns].quantile(0.75)
IQR = Q3 - Q1

# Outlier condition
outliers = ((df[numeric_columns] < (Q1 - 1.5 * IQR)) | (df[numeric_columns] > (Q3 + 1.5 * IQR)))

# Count the number of outliers in each column
outliers_count = outliers.sum()
print("\nNumber of Outliers in Each Numeric Column:")
print(outliers_count)

# Step 6: Handle outliers (we can remove the rows with outliers or cap the outliers)
# For simplicity, we will remove rows with outliers
df_cleaned = df[~outliers.any(axis=1)]

# Step 7: Apply data transformation (log transformation to reduce skewness)
# Let's apply a log transformation to 'chol' (cholesterol level) to reduce skewness

```

```
df_cleaned['log_chol'] = np.log(df_cleaned['chol'] + 1)
```

```
# Step 8: Visualize the distribution of 'chol' before and after transformation
```

```
plt.figure(figsize=(12, 6))
```

```
# Plot before transformation
```

```
plt.subplot(1, 2, 1)
```

```
sns.histplot(df['chol'], kde=True)
```

```
plt.title("Cholesterol Distribution (Before Log Transformation)")
```

```
# Plot after transformation
```

```
plt.subplot(1, 2, 2)
```

```
sns.histplot(df_cleaned['log_chol'], kde=True)
```

```
plt.title("Cholesterol Distribution (After Log Transformation)")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Step 9: Standardize numeric variables
```

```
scaler = StandardScaler()
```

```
df_cleaned[numeric_columns] = scaler.fit_transform(df_cleaned[numeric_columns])
```

```
# Step 10: Show summary of cleaned data
```

```
print("\nSummary of Cleaned Data:")
```

```
print(df_cleaned.describe())
```